



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ingeniería

Compilers Lexer

Student: Cortés Guillén Pavel Francisco - 118001568
Cojulum Ortega Mabel Ximena - 321101921
Gómez Meza Jorge Ángel
Monroy Tapia Jesus Alejandro - 321045979
Villa Garcia Arely Guadalupe - 321000697

Professor: René Adrián Dávila Pérez.

Group: 5

Semester: 2027-1

Deadline: October 6th, 2026

Introduction

The compilation process is fundamentally divided into several phases, with lexical analysis acting as the critical first step. This project presents the design and implementation of a hand-crafted Lexical Analyzer (Lexer) developed in Python. The primary objective of this module is to read raw source code, discard irrelevant characters such as whitespace and comments, and tokenize the input sequence into meaningful symbols.

Unlike automated solutions that rely on tools like FLEX or standard regular expression libraries, this lexer is built entirely from scratch using deterministic finite automaton (DFA) state functions. It is capable of recognizing 40 distinct token types (including keywords, identifiers, operators, constants, and punctuators) while distinguishing case sensitivity. Furthermore, the lexer implements a robust “Panic Mode” error recovery strategy, allowing it to detect and report multiple lexical errors—such as malformed numbers or unclosed string literals—without halting execution. Ultimately, the lexer produces a clean stream of tokens that forms the foundation for subsequent syntax analysis.

Architecture and Project Structure

The architectural design of the lexer follows a modular, layered approach, ensuring separation of concerns and maintainability. The package, structured under `unam.fi.compilers.g5.02`, relies on five core components that coordinate the data flow from raw text to formatted token reports.

- **Main CLI (`main.py`):** Acts as the orchestrator and entry point for the application. It utilizes `argparse` to handle command-line inputs (strings or files) and directs the overall execution flow.
- **Reader (`io/reader.py`):** Responsible for input mechanism abstraction. It safely loads the source code from either direct string inputs or UTF-8 encoded files.
- **Scanner (`lexer/scanner.py`):** The core engine that iterates through the input character by character. It coordinates the DFA states, manages the current line and column pointers, and registers any encountered errors using a Panic Mode recovery mechanism.

- **DFA and Symbol Table** (`lexer/dfa.py` & `symbol_table/symtab.py`): The DFA module implements the State Pattern, using independent functions for each state to identify identifiers, numbers, strings, and operators via a “maximal munch” strategy. The Symbol Table applies early insertion; once a valid word is identified, it is queried to determine if it is a boolean literal, a reserved keyword, or a standard identifier.
- **Writer** (`io/writer.py`): The final output layer that processes the list of valid tokens and lexical errors to generate a structured, human-readable report.

The data flows linearly: raw characters enter the Scanner, which interacts with the DFA and Symbol Table to extract lexemes. These are converted into immutable Token objects and passed to the Writer for final presentation.

Theoretical Foundation

Although the primary scope of this project is the construction of a Lexical Analyzer, its true purpose is to supply a valid stream of tokens to a Parser (Syntax Analyzer). A parser evaluates these tokens against a Context-Free Grammar (CFG) to build a parsing tree, representing the syntactic structure of the program through a series of derivations and tree expansions.

To achieve efficient and deterministic parsing, top-down parsers (such as LL(1) predictive parsers) require grammar formulations that are free of both left recursion and ambiguity. If a grammar exhibits left recursion, a top-down parser can fall into an infinite loop. Similarly, if multiple production rules share common prefixes, the parser cannot determine which derivation to apply based on a single lookahead token. Therefore, grammar transformations like Left Factoring are strictly necessary to guarantee the veracity and performance of the parser relying on our lexer.

Context-Free Grammar (CFG) Proposal

The following CFG excerpts demonstrate the theoretical syntax for variable declarations and print statements within our custom language. The terminal symbols in this grammar directly correspond to the token categories emitted by our Lexer (e.g., `INT`, `ID`, `ASSIGN`, `INTEGER`, `SEMICOLON`).

The initial, unfactored grammar proposal for variable declarations and `printf` statements is defined as follows:

$$\begin{aligned}
\text{Stmt} &\rightarrow \text{int id} = \text{num} ; \\
\text{Stmt} &\rightarrow \text{int id} ; \\
\text{Stmt} &\rightarrow \text{printf} (\text{string}) ; \\
\text{Stmt} &\rightarrow \text{printf} (\text{string} , \text{ArgList}) ;
\end{aligned}$$

Left Factoring Application

In the grammar proposed above, ambiguity arises for an LL(1) parser. Upon reading the `int` token, the parser cannot decide whether it is dealing with a variable assignment or a simple variable declaration without looking further ahead than one token. The same issue occurs with the `printf` token, as both productions begin with the common prefix `printf ( string`.

To resolve this and make the grammar LL(1) compliant, we apply Left Factoring. This process extracts the common prefixes and postpones the parsing decision to a newly generated non-terminal symbol.

Factored Declaration Grammar:

$$\begin{aligned}
\text{Stmt} &\rightarrow \text{int id Rest} \\
\text{Rest} &\rightarrow = \text{num} ; \mid ;
\end{aligned}$$

Factored Print Function Grammar:

$$\begin{aligned}
\text{Stmt} &\rightarrow \text{printf} (\text{string PrintRest} \\
\text{PrintRest} &\rightarrow) ; \mid , \text{ArgList}) ; \\
\text{ArgList} &\rightarrow \text{id ArgList}' \\
\text{ArgList}' &\rightarrow , \text{id ArgList}' \mid \epsilon
\end{aligned}$$

By transforming the grammar using left factoring, we guarantee that every parsing step is deterministic. Our Lexer bridges the gap by strictly enforcing token boundaries (maximal munch) and handling invalid characters beforehand, allowing the LL(1) parser to seamlessly consume the input and perform error-free derivations.